

DOM XSS in `document.write` sink using source `location.search` inside a select element

Summary

A DOM-based Cross-Site Scripting (XSS) vulnerability was identified in the product page's stock-checker functionality.

The application extracts the `storeId` parameter from `location.search` and inserts it into the page using `document.write()`. Because the value is written into an HTML context without appropriate sanitization or encoding, the input can be manipulated to break out of the existing `<select>` element and inject executable HTML/JavaScript.

Vulnerability Classification

Category	Details
Vulnerability	DOM-Based Cross-Site Scripting
Source	<code>location.search</code>
Sink	<code>document.write()</code>
Parameter	<code>storeId</code>
Context	HTML / <code><select></code> element
CWE	CWE-79
OWASP	A03:2021 – Injection

Root Cause

The vulnerable data flow is:

`location.search`



storeId
↓
document.write()
↓
HTML injection
↓
JavaScript execution

The application treats the `storeId` parameter as trusted data and writes it directly into the DOM. Since the value is not safely encoded for its HTML context, an attacker can inject additional markup.

Proof of Concept

First, a random value was supplied through the `storeId` parameter:

```
?productId=1&storeId=test123
```

The value was reflected inside the stock-checker dropdown.

Inspecting the dropdown confirmed that the supplied value was inserted into the `<select>` element.

The following payload was then used:

```
"></select><img src=1 onerror=alert(1)>
```

URL-encoded request:

```
product?productId=1&storeId="></select><img%20src=1%20onerror=alert(1)>
```

When the page was loaded with the modified URL, the injected markup escaped the original `<select>` context and created an image element with an invalid source.

The `onerror` handler subsequently executed:

```
alert(1)
```

The resulting alert confirmed successful DOM XSS.

Payload Breakdown

```
"></select><img src=1 onerror=alert(1)>
```

- `"` — breaks out of the existing HTML attribute context.
- `</select>` — closes the existing select element.
- `<img src=1>` — creates an image element using an invalid image source.
- `onerror=alert(1)` — executes JavaScript when the image fails to load.

Impact

An attacker who can control the affected URL parameter may be able to execute arbitrary JavaScript in the security context of the vulnerable application.

Depending on the application's functionality, successful exploitation could allow an attacker to:

- Manipulate the application's DOM.
- Perform actions through the victim's browser.
- Access sensitive data available to client-side JavaScript.
- Modify displayed content.
- Conduct phishing or UI-manipulation attacks.

The exact impact depends on the application's functionality and security controls.

Remediation

The application should avoid inserting untrusted URL parameters into HTML using `document.write()`.

Recommended mitigations:

- Replace `document.write()` with safer DOM APIs.
- Treat URL parameters as untrusted input.
- Use context-appropriate output encoding.
- Validate `storeId` against the expected input format.
- Use `textContent` when inserting data that should be treated as text rather than HTML.
- Deploy a restrictive Content Security Policy (CSP) as an additional defense-in-depth measure.

Security Takeaway

This vulnerability demonstrates the importance of tracing **source-to-sink data flow** during DOM XSS testing.

Even when an input appears to be a simple URL parameter, passing it directly from `location.search` into a dangerous DOM sink such as `document.write()` can allow an attacker to alter the document structure and execute JavaScript.

Key lesson: Never treat client-controlled URL parameters as trusted data when they reach HTML or JavaScript execution contexts.

References

- PortSwigger Web Security Academy — DOM-based XSS
- OWASP — Cross-Site Scripting (XSS)
- CWE-79 — Improper Neutralization of Input During Web Page Generation